Certificate Renewal Automation with Claude: Research and Content Plan

Network and Infrastructure Automation · 3 days (8 hours per day) · Intermediate · Compiled 30 October 2026

This document is the working plan for building the course from its outline. It records where the course came from, why the outline is shaped the way it is, what goes into every module (lessons, teaching points, demo, lab, pitfalls and knowledge checks), the lab environment to prepare, and the sources behind each fact. Numbers in square brackets point to the reference list at the end; every URL there was opened and checked against the fact it supports.

- 1. Where this course came from
- 2. How the outline was shaped
- 3. Audience and prerequisites
- 4. Lab environment
- 5. Module-by-module content plan
- 6. Capstone plan
- 7. Promotional video: research behind the script
- References

1. Where this course came from

Client input.

> Client use case 1, 'Certificate Update Automation': "Certificate update by automation for all kinds of load balancer devices: F5, NetScaler, Radware, Anaya LB." The client's stated aim for all use cases is "to do automation using Claude for all these content" with titles on the line of "Automation of <> using Claude".

Expired certificates are one of the most avoidable causes of outages on load balancers, and renewal is about to become much more frequent. The CA/Browser Forum Baseline Requirements cut the maximum validity of publicly trusted TLS certificates from 398 days to 200 days in March 2026, 100 days in March 2027 and 47 days in March 2029 [18]. A team that renews a few hundred certificates by hand once a year will soon be doing it several times a year on every load balancer they own.

The hard part is not getting a certificate; ACME already automates issuance [5]. The hard part is putting it on the right objects on each platform: F5 certificate and key files referenced from client-ssl profiles [1][10], NetScaler cert-key pairs bound to every SSL virtual server [3], Alteon certificates applied through a separate step [4], and HA pairs that must be synchronized [13]. Each platform has its own API and its own failure modes.

This course teaches teams to use Claude where it helps most (reading inventory, drafting plans and code, explaining) while keeping every device write in deterministic, tested code behind a human approval. That split follows OWASP guidance on limiting agent permissions and requiring human approval for high-impact actions [28], and it gives the client a pipeline they can audit.

2. How the outline was shaped

Decision	Rationale
Cover F5, NetScaler and Radware in depth and Anaya LB through an adapter	The client named all four. F5, NetScaler and Radware have public API documentation [1][2][29]; we could not find public documentation for Anaya LB, so the course builds a shared adapter contract that the client can implement against their own Anaya documentation.
Claude plans and drafts; deterministic code executes	Certificate installs touch production traffic. Keeping writes in tested Python, with the model limited to read-only tools, follows OWASP's excessive-agency mitigations [28] and makes behavior repeatable.
Start with a manual renewal before any automation	Learners need to see where outages come from (chain, key match, missed binding, HA sync) before they can write checks that catch them.
ServiceNow change records and approval as the gate	The client's other use cases already center on ServiceNow (SNOW). Using the Table API [24] and Flow Designer approval [25] fits existing change practice.
Keys and CSRs generated by code, never by the model	Private keys must never enter prompts or logs. Generating CSRs in code with a standard library [27] keeps key material out of the model's context.
GitHub pull requests and Actions for review and CI	Reviewing renewal plans like code gives a second human view and a test record; scheduling through Actions [26] gives a simple way to run inventory daily.

3. Audience and prerequisites

Role	What they need	Starting point
Load balancer and ADC engineers	A safe, repeatable way to renew certificates across platforms	Strong on one platform's GUI and CLI; limited API and Python experience
PKI and certificate owners	Visibility of where each certificate is bound and proof of renewal	Strong on certificates and CAs; limited load balancer knowledge
Network automation leads	A pattern for using an AI assistant safely in production change	Comfortable with Python, Git and REST APIs
Enterprise IT and network operations staff	To run and support the pipeline day to day	Familiar with change management and ServiceNow

Prerequisites from the outline

- Working knowledge of at least one load balancer platform (F5, NetScaler, Radware or similar)
- TLS basics: keys, certificates, chains, SANs and expiry
- Basic Python and Git
- Familiarity with an AI coding assistant is helpful, not required

Gemoor Tech
For more info - https://gemoor.com

Checks before enrolment

- Can you find which virtual servers use a given certificate on at least one platform you manage?
- Can you explain the difference between a leaf certificate, an intermediate and a root?
- Can you read and run a short Python script and make a commit in Git?
- Can you call a REST API with curl and read a JSON response?
- Do you have, or can you get, a laptop that can reach the lab over SSH and HTTPS?

Course outcomes

- Explain the certificate lifecycle as each load balancer platform implements it and where renewal outages really come from
- Set up Claude Code, the Claude Agent SDK and an MCP server as a safe workbench for certificate operations
- Build a typed certificate inventory from F5 iControl REST, NetScaler NITRO, Radware Alteon and other load balancers (Anaya LB)
- Use a Claude planning agent to draft renewal plans as structured JSON and open ServiceNow change records with citations to inventory evidence
- Write deterministic, pytest-covered executors that upload, install, bind and verify certificates per platform
- Enforce guardrails, human approval, pre- and post-checks, automatic rollback and an audit trail reviewable through GitHub pull requests and Actions

4. Lab environment

Each learner gets a small estate: an F5 BIG-IP VE pair, a NetScaler VPX, an Alteon VA, a step-ca ACME lab CA, a ServiceNow developer instance and a GitHub repository. All certificates are issued by the lab CA for example.corp names. Recorded API responses let every test run offline.

Components

GenNoorTech
For more info - https://gennoor.com

Component	Version or edition	Purpose	How to obtain	Notes
F5 BIG-IP Virtual Edition (HA pair lb-f5-01 and lb-f5-02)	BIG-IP 17.5.x	F5 inventory and executor labs	30-day trial keys through MyF5, or F5 lab licenses through a reseller	Trial eligibility varies by account; request keys at least two weeks before the course.
NetScaler VPX (lb-ns-01)	NetScaler 14.1, VPX Express	NetScaler inventory and executor labs	VPX Express needs no license file [32]	Express is limited to 20 Mbps and 250 SSL sessions, which is enough for the labs [32].
Radware Alteon VA (lb-rw-01)	Alteon 33.x (alteon-sdk requires 31.0.10.0 or 32.2.2.0 minimum) [29]	Alteon inventory and executor labs	Evaluation license through Radware sales or partner	We could not confirm public trial terms; if no VA is available, run Alteon labs against the recorded responses only.
Anaya LB	Client-supplied	Adapter stub and contract tests	Client to provide API documentation or a test device	No public documentation was found; the course ships a stub.
step-ca ACME lab CA	Current open-source release	Issue lab certificates through ACME	Open source from Smallstep; enable ACME with step ca provisioner add acme --type ACME [31]	Gives learners real ACME orders without public DNS.
ServiceNow Personal Developer Instance	Current release	Change records, CMDB owner data and approvals	Free through the ServiceNow Developer Program [33]	Instances unused for ten days are reclaimed [33]; provision close to the course date and export the flow.
Claude Code and Claude Agent SDK	Current release	Workbench, planning agent and drafting	Install Claude Code with the official installer [8]; Agent SDK from the package index with an API key [9]	Use organization API keys, not personal logins.
Python, pytest, pyca/cryptography	Python 3.11 or later	Collectors, executors and tests	Standard package index	Pin versions in requirements.txt.

Gemnoor Tech TRAINING INNOVATION For more info - http://gemnoor.com

GitHub repository with Actions	GitHub.com or Enterprise	Pull request pipeline and scheduling	One repository per team	Use a self-hosted runner inside the lab network for live device steps.

Fixtures and sample data

Fixture	Contents	Used in
Recorded API responses	F5 ssl-cert, ssl-key, client-ssl, virtual server; NITRO sslcertkey, bindings, systemfile; Alteon SDK reads; including error cases	Certificate inventory and expiry discovery across platforms; both executor modules
CMDB export	Owners for most lab certificates, with deliberate gaps	Certificate inventory and expiry discovery across platforms; Renewal planning agent and ServiceNow change records
Broken certificate set	Leaf without intermediate, mismatched key, wrong SAN	Pre- and post-checks, rollback and the GitHub pipeline
Change records	Approved, unapproved and expired-window lab changes	Guardrails, approvals and audit for certificate changes

Lab safety rules

- Lab devices only; no production addresses in any config file
- All certificates come from the lab CA for example.corp names
- Device credentials only in environment variables or the lab vault, never in prompts or the repository
- The MCP server exposes read-only tools; device writes only through executors with an approved change
- Reset scripts restore every device to its baseline at the end of each day

5. Module-by-module content plan

Day 1

Module 1: Certificate lifecycle on load balancers and the automation baseline

Goal. Learners understand exactly where a certificate lives and is bound on each platform, and where manual renewals fail, before any automation is written.

Outline topics.

- How each platform stores and binds certificates: F5 client-ssl profiles and virtual servers, NetScaler sslcertkey and sslvserver bindings, Radware Alteon certificate repository and SSL

Genmoor Tech
For more info: https://genmoor.com

policies, other load balancers (Anaya LB)

- Where renewal outages come from: wrong chain, mismatched key, binding missed on a standby, HA sync skipped
- The operating pattern for the whole course: Claude plans, drafts and explains; deterministic code executes; a human approves; everything is logged
- Lab estate walkthrough: lb-f5-01, lb-ns-01, lb-rw-01, an ACME lab CA and a ServiceNow developer instance

Learning objectives.

- Learners can trace a certificate from file object to virtual server on F5, NetScaler and Alteon
- Learners can name the four most common renewal outage causes and show where each happens
- Learners can explain why shorter certificate lifetimes make manual renewal unsustainable
- Learners can state the course operating pattern: Claude plans, code executes, a human approves, everything is logged

F5 BIG-IP: files, profiles, virtual servers

- Certificates and keys are separate file objects: /mgmt/tm/sys/file/ssl-cert and /mgmt/tm/sys/file/ssl-key, each keyed by name, partition and subPath [1][14]
- A client-ssl profile references the pair through certKeyChain; the older cert and chain properties are deprecated [10]
- A client-ssl profile needs at least one cert/key pair; tmsh edits it with cert-key-chain add, modify or replace-all-with [11]
- The profile, not the certificate, is attached to the virtual server, so one certificate can sit behind many virtual servers through shared profiles
- Every ssl-cert object exposes expirationDate, subject, issuer, subjectAlternativeName and serialNumber, which is what inventory will read [1]

NetScaler: cert-key pairs and vserver bindings

- Files sit in /nsconfig/ssl by default; add ssl certKey creates a named pair that points at the files [3]
- Intermediates must be linked to the server cert-key pair before binding (linkcertkeyname in NITRO) [3][2]
- Each SSL virtual server needs its own binding: bind ssl vserver <vs> -certkeyName <name>, checked with show ssl vserver [3]
- Built-in expiry monitoring exists (expirymonitor, notificationperiod 10 to 100 days) but only alerts; it never renews [2]
- Updating a pair in place (update ssl certKey, NITRO ?action=update) swaps the files under every existing binding at once [2]

Radware Alteon and other platforms (Anaya LB)

- Alteon CLI installation: one PEM with server cert then intermediates, entered under cfg then cert, terminated with three periods, then apply [4]
- Configuration changes are pending until applied; treat apply and save as explicit, separate steps in every runbook
- Radware publishes a Python SDK over the Alteon REST backend with an SSL key configurator, which the course uses as the API reference point [29]
- Anaya LB has no public API documentation we could verify; the course treats it through an adapter that the client fills in against their own device documentation

Where renewal outages really come from

- Wrong or missing chain: the leaf is valid but clients cannot build a path; openssl s_client -showcerts shows what the server really sends [20]
- Key and certificate mismatch: the object installs but the handshake fails
- Binding missed on one of several virtual servers or SNI profiles, especially where sniDefault decides the fallback profile [10]
- HA pair not synchronized after the change, so the standby still serves the old certificate after failover [13]
- Public certificate lifetimes drop to 200 days from March 2026, 100 days from March 2027 and 47 days from March 2029, so renewals become frequent routine work [18]

The operating pattern and the lab estate

- Claude reads, plans, drafts code and explains; deterministic Python performs every device write
- Every write needs an approved change and a named human approver; the agent cannot self-approve [28]
- Lab estate: lb-f5-01 (with a standby peer), lb-ns-01, lb-rw-01, a step-ca ACME lab CA and a ServiceNow developer instance [31][33]
- Everything is logged: prompts, tool calls, device requests and responses, approver and time

Instructor demo.

1. Open the F5 GUI and REST side by side; GET /mgmt/tm/ltm/profile/client-ssl/~Common~clientssl_payments and point to certKeyChain
2. GET the referenced ssl-cert object and read expirationString, serialNumber and subjectAlternativeName aloud
3. On lb-ns-01 run show ssl certKey and show ssl vserver vs_portal_443 to show the binding and linked intermediate
4. Run openssl s_client -connect portal.example.corp:443 -servername portal.example.corp -showcerts against both devices and compare chains
5. Renew one F5 certificate by hand and deliberately skip config sync; fail over and show the old serial being served
6. Draw the failure-point map on the whiteboard as learners call out each step

Hands-on lab. Renew one certificate by hand on lb-f5-01 and one on lb-ns-01 using a lab CA certificate, writing down every step and every point where a mistake would cause an outage.

1. Request a certificate for pay.example.corp from the step-ca lab CA using the CSR provided
2. On F5, import the key and certificate with date-suffixed names through the GUI or tmsh
3. Update clientssl_payments cert-key-chain to the new pair and confirm with tmsh list ltm profile client-ssl clientssl_payments
4. Run tmsh run cm config-sync to-group <device group> and confirm the peer shows in sync
5. On NetScaler, copy files to /nsconfig/ssl, add the cert-key pair, link the intermediate and bind it to vs_portal_443
6. Verify both with openssl s_client -showcerts and record the new serial numbers
7. Write each step into the runbook template with the time it took
8. Mark every step on the failure-point map with what would go wrong and how you would detect it

Lab success criteria.

- Both virtual servers serve the new serial with a complete chain
- The F5 standby holds the same certificate objects after sync
- The runbook lists every step, command and verification
- The failure-point map covers chain, key match, missed binding and HA sync at minimum

Deliverable. Manual renewal runbook and failure-point map.

Common pitfalls.

- Overwriting the existing certificate object instead of creating a new date-suffixed object, which removes the rollback path
- Forgetting to link the intermediate on NetScaler before binding
- Checking only the active F5 unit and not the standby
- Trusting the browser view of the chain, which may fill gaps from its own cache, instead of s_client -showcerts

Knowledge check.

1. On F5, what attaches a certificate to a virtual server? Answer: A client-ssl profile whose certKeyChain references the cert and key objects; the profile is then attached to the virtual server.
2. Why does NetScaler expiry monitoring not solve the renewal problem? Answer: expirymonitor and notificationperiod only raise an alert before expiry; they do not request, install or bind a new certificate.
3. What does openssl s_client -showcerts show that a browser may not? Answer: Only the certificates the server actually sent, in order, without filling in missing intermediates.
4. Why will renewal frequency increase over the next few years? Answer: The CA/Browser Forum Baseline Requirements cut maximum public certificate validity to 200 days in 2026, 100 days in 2027 and 47 days in 2029.

Gennoor Tech
For more info - https://gennoor.com

Slides and assets to prepare.

- Side-by-side diagram of the object model on F5, NetScaler and Alteon
- Failure-point map template (blank and instructor-completed versions)
- Manual renewal runbook template
- Validity reduction timeline chart from the Baseline Requirements
- Lab estate topology diagram with hostnames

Sources for this module. 1, 2, 3, 4, 10, 11, 13, 14, 18, 20, 28, 29, 31, 33

Module 2: Setting up Claude Code, the Agent SDK and MCP for certificate work

Goal. Learners build a workbench where Claude can read certificate data through narrow read-only tools and cannot change any device.

Outline topics.

- Claude Code project setup: CLAUDE.md with estate conventions, naming rules for certificate objects, and what the agent may never do
- Claude Agent SDK in Python: defining tools with JSON schemas, forcing structured JSON outputs, handling tool-use loops
- An MCP server exposing read-only inventory tools (list_certs, get_expiry, get_bindings) and no write tools
- Secrets handling: device credentials in environment variables and a vault, never in prompts or repositories

Learning objectives.

- Learners can install Claude Code and write a project CLAUDE.md that encodes estate conventions and red lines
- Learners can define typed tools with the Agent SDK and expose them through an MCP server
- Learners can block unwanted tool calls with permissions and PreToolUse hooks
- Learners can keep device credentials out of prompts, logs and the repository

Claude Code and the project file

- Install with curl -fsSL https://claude.ai/install.sh | bash on macOS, Linux or WSL; claude -p runs it non-interactively for scripts and CI [8]
- Project instructions live in ./CLAUDE.md or ./.claude/CLAUDE.md and are shared through source control [30]
- Put naming rules in CLAUDE.md: <fqdn>_<YYYYMMDD> for certificate objects, clientssl_<app> for profiles
- State red lines plainly: never call write endpoints, never read files under secrets/, never print key material
- Keep CLAUDE.md short and specific; long files cost context and reduce adherence [30]

Agent SDK tools with schemas

- Define each tool with @tool (name, description, input schema, handler), wrap them with create_sdk_mcp_server and pass them in mcp_servers [22]
- Tool names become mcp__certops__list_certs and must be listed in allowed_tools to run without a prompt [22]
- Mark read tools with readOnlyHint and return machine-readable results in structuredContent alongside text [22]
- Return isError: true with a clear message when a device call fails so the model does not invent data [22]
- The SDK uses API-key authentication for anything you ship to others [9]

A read-only certificate MCP server

- Three tools only: list_certs(device), get_expiry(device, cert), get_bindings(device, cert)
- No write tool exists in the server, so a prompt cannot reach a write path
- MCP standardizes how tools are described and called, so the same server works from Claude Code and the Agent SDK [7]
- Each tool returns typed records validated before they are returned

Guardrails at the workbench

- PreToolUse hooks run before a tool call and can deny it with a reason; use one to block any Bash call containing curl -X POST, PUT, PATCH or DELETE against device addresses [21]
- Credentials come from environment variables injected from a vault at process start, never from prompts
- Redact Authorization, Cookie and X-NITRO-PASS headers in every log line [2]
- Least privilege on devices too: the read tools use a read-only device account [28]

Instructor demo.

1. Create the repo, run claude in it and write CLAUDE.md live with the naming rules and red lines
2. Write list_certs for F5 with @tool, returning name, expirationDate and serialNumber from /mgmt/tm/sys/file/ssl-cert
3. Register the server and show the tool name mcp__certops__list_certs in the session
4. Ask Claude to explain bindings on lb-f5-01 and show it calling only read tools
5. Ask Claude to replace the certificate and show the request refused because no write tool exists and the hook denies device writes
6. Show the log file with secrets redacted

Hands-on lab. Build the read-only certificate MCP server, connect it to Claude Code and ask Claude to explain the bindings on lb-f5-01 without changing anything.

1. Clone the starter repo and create a Python virtual environment
2. Write CLAUDE.md with naming conventions, lab hostnames and red lines

3. Implement list_certs, get_expiry and get_bindings for F5 against the recorded responses in fixtures/f5/
4. Add readOnlyHint to every tool and return structuredContent
5. Register the server in Claude Code and confirm the three tools appear
6. Add a PreToolUse hook that denies any shell command sending POST, PUT, PATCH or DELETE to a device address
7. Point the tools at lb-f5-01 using credentials from environment variables
8. Ask Claude which virtual servers depend on pay.example.corp and save the answer

Lab success criteria.

- Claude answers using only tool results and names the profile and virtual servers correctly
- A request to change the certificate is refused and the refusal is logged
- No credential appears in the repo, the transcript or the log
- pytest passes for all three tools against recorded responses

Deliverable. Working agent workbench with read-only MCP tools.

Common pitfalls.

- Exposing a generic run_command tool, which turns a read-only server into a write path
- Putting the device password in CLAUDE.md or a prompt 'just for the lab'
- Returning free text from tools so the model has to guess at field meanings
- Relying on tool annotations alone; MCP treats them as hints, so enforcement must be in code and hooks

Knowledge check.

1. What makes a tool callable without a prompt in the Agent SDK? Answer: Listing its full name, mcp__{server}__{tool}, in allowed_tools.
2. Where should the rule 'never call write endpoints' be enforced? Answer: In code: no write tools in the server plus a PreToolUse hook; CLAUDE.md states the rule but is not enforcement.
3. What should a tool return when the device call fails? Answer: isError: true with a clear message, so Claude reports the failure instead of inventing data.

Slides and assets to prepare.

- Starter repo with CLAUDE.md template and tool skeletons
- Diagram: Claude Code to MCP server to device, with the read-only boundary marked
- Hook script example that denies device write methods
- Recorded F5 responses for list, expiry and bindings

Sources for this module. 2, 7, 8, 9, 21, 22, 28, 30

GenNoorTech TRAINING INNOVATE BUILD For more info - https://gennoor.com

Module 3: Certificate inventory and expiry discovery across platforms

Goal. Learners build collectors that turn three different APIs into one typed inventory, with unknowns marked rather than guessed.

Outline topics.

- F5 iControl REST: /mgmt/tm/sys/file/ssl-cert, /mgmt/tm/ltm/profile/client-ssl and virtual server lookups; tmsh equivalents for verification
- NetScaler NITRO: /nitro/v1/config/sslcertkey and sslvserver_sslcertkey_binding; Radware Alteon REST API certificate and SSL policy objects; an adapter interface for other load balancers (Anaya LB)
- A typed inventory record: device, platform, certificate name, subject and SANs, chain, expiry, bound virtual servers, owner, confidence
- Claude drafts the collectors; you review the diffs and run pytest against recorded API responses

Learning objectives.

- Learners can read certificates, profiles and bindings from F5 iControl REST and NetScaler NITRO
- Learners can map Alteon certificate data through the Radware SDK or REST backend behind an adapter
- Learners can define a typed inventory record with a confidence field
- Learners can review Claude-drafted collector code and test it against recorded responses

F5 collection

- GET /mgmt/tm/sys/file/ssl-cert returns every certificate with expirationDate, subjectAlternativeName, issuer and serialNumber [1]
- GET /mgmt/tm/ltm/profile/client-ssl returns certKeyChain entries that map profiles to cert and key names [10]
- Map profiles to virtual servers by reading each virtual server's profiles list; verify a sample with tmsh list ltm profile client-ssl [11]
- Collect partitions explicitly; objects outside /Common are easy to miss

NetScaler collection

- Log in with POST /nitro/v1/config/login and send Cookie: NITRO_AUTH_TOKEN on later calls [2]
- GET /nitro/v1/config/sslcertkey returns pairs with daystoexpiration and linked CA pair names [2]
- GET /nitro/v1/config/sslvserver_sslcertkey_binding/<vservername> returns the certkeyname and whether it is an SNI or CA binding [15]
- Use ?count=yes to check totals before paging through large estates [2]

Alteon and the adapter interface

GemnoorTech TRAINING INNOVATE BUILD For more info - https://gemnoor.com

- The Radware alteon-sdk reads SSL keys and certificates through configurators over the REST backend; it requires Alteon 31.0.10.0 or 32.2.2.0 at minimum [29]
- Wrap every platform behind one interface: list_certificates(), list_bindings(), device_facts()
- The Anaya LB adapter is a stub that returns UNKNOWN for every field until the client supplies its API details
- Adapters normalize dates to UTC and serials to uppercase hex so rows compare across platforms

The typed inventory record

- Fields: device, platform, cert_name, subject, sans, issuer, serial, not_after, chain_complete, bound_to, owner, confidence
- confidence is high when the binding was read from the device, low when inferred, unknown when the source is missing
- Owner comes from CMDB; a missing owner is recorded as UNKNOWN, never filled in by the model
- Every row carries an ID such as inventory#f5-0031 so later plans can cite it

Claude drafts, you review

- Ask Claude to draft each collector from the API reference and a recorded response
- Review the diff line by line; reject any write call or unbounded retry
- Test with pytest against recorded JSON; monkeypatch the HTTP client so tests never touch a device
- Ask Claude to explain each field mapping and compare the explanation to the API reference

Instructor demo.

1. Show raw JSON from F5 ssl-cert, client-ssl and a virtual server, and the joins needed between them
2. Show raw NITRO sslcertkey and sslvserver_sslcertkey_binding responses for lb-ns-01
3. Prompt Claude to draft the NetScaler collector and review the diff aloud, rejecting one unsafe change
4. Run pytest against recorded responses and show a failing test caught by a missing field
5. Generate the expiry report and point out rows marked UNKNOWN

Hands-on lab. Build the multi-platform inventory collector and produce one expiry report for the lab estate with unknowns marked explicitly.

1. Define the inventory record as a dataclass or Pydantic model with the fields above
2. Ask Claude to draft the F5 collector; review and commit it
3. Ask Claude to draft the NetScaler collector including login and bindings; review and commit it
4. Implement the Alteon adapter against recorded responses
5. Add the Anaya LB stub that returns UNKNOWN
6. Write pytest cases for each collector with recorded responses
7. Join owner data from the ServiceNow CMDB export fixture

Germoor Tech TRAIN INNOVATE BUILD For more info - https://germoor.com

8. Produce expiry_report.csv and expiry_report.md sorted by not_after

Lab success criteria.

- Every certificate on the three lab devices appears once with its bindings
- Rows with a missing owner or unverified binding are marked UNKNOWN or low confidence
- All collector tests pass without network access
- The report shows pay.example.corp bound to vs_payments_443 on lb-f5-01

Deliverable. Certificate inventory and expiry report.

Common pitfalls.

- Assuming certificate name equals hostname; match on SANs instead
- Missing SNI bindings or non-Common partitions
- Letting the model fill in a missing owner from the hostname
- Tests that call a live device and pass only when the lab is up

Knowledge check.

1. Which NITRO resource tells you which virtual servers use a cert-key pair? Answer: sslvserver_sslcertkey_binding, read per virtual server.
2. Why does each inventory row carry an ID? Answer: So the planning agent can cite exact rows as evidence and reviewers can check them.
3. What should the collector do when it cannot read a binding? Answer: Record the field as unknown with low confidence; never infer it silently.

Slides and assets to prepare.

- Field mapping table from each platform API to the inventory record
- Recorded API responses for all three lab devices
- Sample expiry report with UNKNOWN rows highlighted
- pytest fixtures and monkeypatch examples

Sources for this module. 1, 2, 10, 11, 15, 29

Day 2

Module 4: Renewal planning agent and ServiceNow change records

Goal. Learners build an agent that turns inventory into a cited JSON renewal plan and a ServiceNow change, with code, not the model, handling keys and CSRs.

Outline topics.

- Planning agent: group certificates by expiry window, owner and platform; propose order, maintenance window and blast radius as structured JSON

GenNoor Tech TRAINING INNOVATE BUILD For more info - https://gennoor.com

- Mandatory citations to inventory rows; unknowns (missing owner, unverified binding) block the plan instead of being guessed
- Opening and updating change_request via the ServiceNow Table API; triggering an approval path in Flow Designer
- Requesting the new certificate from the ACME lab CA or an internal CA request with CSR generated by code, not by the model

Learning objectives.

- Learners can force the planning agent to return a plan that matches a JSON schema
- Learners can make unknowns block a plan rather than be guessed
- Learners can create and update change_request records through the Table API and route them to approval
- Learners can generate keys and CSRs in code and obtain certificates from an ACME lab CA

Plan schema and structured output

- Define the plan schema: steps[], each with device, cert, target_objects, window, blast_radius, evidence[] and rollback
- Send the schema in output_config.format so the response matches it; use strict tool use for tool inputs [23]
- Validate the plan again in code before anything else reads it
- Order by expiry first, then blast radius (number of virtual servers and business service)

Citations and blocking unknowns

- Every step must cite inventory row IDs; the validator rejects steps without evidence
- Any cited row with owner UNKNOWN or binding confidence low turns the plan status to BLOCKED with a reason
- The agent may explain why a plan is blocked; it may not change the inventory to unblock it
- A blocked plan is still saved and shown, so the owner gap becomes a visible task rather than a silent skip

ServiceNow change records

- Create with POST /api/now/table/change_request (one record per call, 201 on success) [24]
- Update with PATCH /api/now/table/change_request/{sys_id}; read state with GET and sysparm_fields to keep payloads small [24]
- Put the plan summary, evidence row IDs and rollback command in the description and work notes
- Use a Flow Designer flow with Ask for Approval on the change record; read the resulting approval state rather than assuming it [25]
- Give the approval a due date so a forgotten request resolves instead of waiting forever [25]

Keys, CSRs and the ACME lab CA

- Generate the key and CSR in the executor with x509.CertificateSigningRequestBuilder; the CA gets the CSR, never the key [27]
- ACME flow: order, prove control with a challenge, finalize with the CSR, download the certificate [5]
- In the lab, step-ca acts as the ACME CA after step ca provisioner add acme --type ACME [31]
- For internal CAs without ACME, the executor submits the CSR to the CA's request API and stores the returned chain
- The model never sees key material; it only sees the CSR subject, SANs and the returned certificate serial

Instructor demo.

1. Show the plan JSON schema and run the agent on the inventory fixture
2. Show a plan step citing inventory#f5-0031 and cmdb#9921
3. Remove the owner from one row and rerun to show the plan BLOCKED with a reason
4. Create the change through the Table API and show it in the ServiceNow developer instance
5. Trigger the approval flow and show the approval state read back by code
6. Run the ACME request from code and show the key file permissions and the absence of key data in the transcript

Hands-on lab. Run the planning agent on the inventory, inspect the JSON plan, and watch it open a change record that cites the exact certificates it will touch.

1. Write the plan JSON schema and a validator
2. Implement the planning agent with the inventory tools and structured output
3. Run it on the lab inventory and save plan.json
4. Add the rule that UNKNOWN owner or low-confidence binding blocks the step, and test it
5. Implement create_change and update_change against the Table API
6. Build or import the Flow Designer approval flow for the lab change model
7. Implement request_certificate with key and CSR generation in code and ACME issuance from step-ca
8. Attach the plan JSON to the change record

Lab success criteria.

- plan.json validates against the schema and every step cites inventory rows
- A row with a missing owner blocks its step with a logged reason
- The change record exists with plan summary, evidence and rollback text
- The issued certificate matches the CSR and no key material appears in any log or transcript

Deliverable. Renewal planning agent and change-record integration.

Common pitfalls.

- Letting the model write the CSR or handle the key 'to save time'
- Assuming the change is approved because the API call succeeded

GennoorTech
For more info: https://gennoor.com

- Free-text plans that cannot be validated
- Hard-coding change state numbers without checking the instance's change model

Knowledge check.

1. How does the course guarantee the plan's shape? Answer: A JSON schema sent through structured outputs, then validated again in code.
2. What should happen if a cited inventory row has no owner? Answer: The step is blocked with a reason; the agent does not guess the owner.
3. Which ACME step uses the CSR? Answer: Finalizing the order, after the identifier challenges are satisfied.

Slides and assets to prepare.

- Plan JSON schema and example plans (clean and blocked)
- ServiceNow change request field map
- Flow Designer approval flow export for the lab
- ACME sequence diagram

Sources for this module. 5, 23, 24, 25, 27, 31

Module 5: Deterministic executor: F5 BIG-IP with iControl REST and tmsh

Goal. Learners build and test an F5 executor that installs a new certificate as new objects, swaps the profile, syncs the HA pair and verifies, with no model in the write path.

Outline topics.

- Upload key and certificate through /mgmt/shared/file-transfer/uploads, then create sys file ssl-key and ssl-cert objects
- Update the client-ssl profile cert-key-chain and confirm virtual server bindings; verify with tmsh list ltm profile client-ssl
- Idempotency, naming with date suffixes, and config sync across the HA pair before declaring success
- pytest with recorded iControl responses; Claude drafts code, explains each call, and never holds the credentials

Learning objectives.

- Learners can upload files to BIG-IP through the file-transfer worker with chunking
- Learners can create ssl-key and ssl-cert objects and update a client-ssl profile's certKeyChain
- Learners can make the executor idempotent and HA-aware
- Learners can test the executor against recorded iControl responses

Uploading files

- POST /mgmt/shared/file-transfer/uploads/<filename> writes to /var/config/rest/downloads/ [12]

Gennoor Tech
PRIVATE. BUILT. INNOVATE.
For more info - https://gennoor.com

- Send Content-Range: start-end/filesize; files over 1M must be sent in chunks [12]
- Upload the key and certificate as separate files and remove them from the downloads directory after import
- Check the upload response for the final byte count before creating objects

Creating objects and swapping the profile

- Create the key at /mgmt/tm/sys/file/ssl-key and the certificate at /mgmt/tm/sys/file/ssl-cert with sourcePath file:/var/config/rest/downloads/<file> [14][1]
- Name objects <fqdn>_<YYYYMMDD> so the previous object stays for rollback
- PATCH /mgmt/tm/ltm/profile/client-ssl/~Common~<profile> with the new certKeyChain entry (cert, key, chain) [10]
- Verify with tmsh list ltm profile client-ssl <profile> cert-key-chain [11]

Idempotency and HA

- Before each write, read the current state; if the target object already exists with the same serial, skip
- Record state after each step so a rerun continues from the last completed step
- Run cm config-sync to-group for the device group and read sync status before reporting success [13]
- Never write to the standby; writes go to the active unit and reach the peer through sync

Testing and the model's role

- pytest with recorded iControl responses for every call, including error responses
- Contract tests: a plan step in, a result with request IDs and evidence out
- Claude drafts and explains each call; the executor runs under a service account whose credentials Claude never receives
- Dry-run mode prints the exact method, URI and redacted body for every planned call

Instructor demo.

1. Walk through the executor sequence diagram: upload, create key, create cert, patch profile, sync, verify
2. Run the upload with a 3 MB test file to show chunking
3. Run the executor in dry-run mode and show the planned API calls
4. Run it live against lb-f5-01 and show the new objects and profile
5. Rerun the same plan and show every step skipped
6. Show tmsh output on both units after sync

Hands-on lab. Build and test the F5 executor against a mock iControl endpoint, then run it against lb-f5-01 in dry-run and live modes.

1. Implement chunked upload with Content-Range

2. Implement create_key, create_cert and patch_profile using the recorded responses as the contract
3. Add read-before-write idempotency checks
4. Add config sync and sync-status verification
5. Write pytest cases, including a failed upload and a 409 conflict
6. Run dry-run against lb-f5-01 and review the planned calls
7. Run live with an approved lab change ID
8. Rerun to prove idempotency

Lab success criteria.

- All tests pass offline
- The profile references the new date-suffixed objects and the old objects remain
- Both units are in sync after the run
- A second run changes nothing

Deliverable. Tested F5 certificate executor.

Common pitfalls.

- Uploading without Content-Range or in one chunk over the size limit
- Patching the deprecated cert and chain properties instead of certKeyChain
- Declaring success before config sync completes
- Deleting the old certificate object in the same run

Knowledge check.

1. Where do files land after POST /mgmt/shared/file-transfer/uploads/? Answer: /var/config/rest/downloads/.
2. Why use date-suffixed object names? Answer: So the previous object stays in place as the rollback target and reruns are easy to detect.
3. When is an F5 renewal finished? Answer: When the profile references the new objects, config sync to the group has completed and post-checks pass.

Slides and assets to prepare.

- F5 executor sequence diagram
- Recorded iControl responses including error cases
- Mock iControl server for the lab
- Checklist of F5 verification commands

Sources for this module. 1, 10, 11, 12, 13, 14

GenNoorTech
For more info: https://gennoor.com

Module 6: Deterministic executors: NetScaler NITRO, Radware Alteon and other load balancers (Anaya LB)

Goal. Learners implement NetScaler and Alteon executors and an adapter stub that all pass one shared contract test suite.

Outline topics.

- NITRO API: upload with systemfile, add or update sslcertkey, bind to sslvserver, save config; handling HA primary-only writes
- Radware Alteon: certificate import and SSL policy update through the Alteon REST API, with apply and save as explicit steps
- Adapter pattern for other load balancers (Anaya LB): one interface, one contract test suite, platform-specific implementation
- A common executor contract: plan in, result and evidence out, every call logged with request and response IDs

Learning objectives.

- Learners can upload, add or update and bind a cert-key pair through NITRO and save the configuration
- Learners can apply and save an Alteon certificate change as explicit steps
- Learners can define a common executor contract and run the same tests against every platform
- Learners can log every call with request and response identifiers

NetScaler through NITRO

- Upload files with POST /nitro/v1/config/systemfile using Base64 filecontent and filelocation /nsconfig/ssl [16][3]
- New pair: POST /nitro/v1/config/sslcertkey with certkey, cert and key; link the intermediate with linkcertkeyname [2]
- In-place renewal: POST /nitro/v1/config/sslcertkey?action=update replaces the files under every existing binding [2]
- New binding: PUT /nitro/v1/config/sslvserver_sslcertkey_binding with vservername and certkeyname [15]
- Save with POST /nitro/v1/config/nsconfig?action=save; write only to the HA primary [17]

Radware Alteon

- Import server certificate and chain in one PEM, then apply, as in the CLI flow [4]
- Through the SDK, deploy changes with dry_run and get_diff first, then commit through management config functions [29]
- Treat apply and save as separate executor steps with their own checks
- Confirm the SSL policy references the new certificate before declaring success

GenNoor Tech TRAINING INNOVATE BUILD For more info - https://gennoor.com

The adapter for other load balancers (Anaya LB)

- One interface: upload(), install(), bind(), save(), verify(), rollback()
- One contract test suite that every adapter must pass, using recorded or mocked responses
- The Anaya LB adapter raises NotImplemented on every write until its API is documented, and the contract tests mark it expected-fail
- Platform quirks (apply steps, primary-only writes, chunked uploads) stay inside the adapter, never in the pipeline

The executor contract and logging

- Input: a validated plan step plus an approved change ID
- Output: status, before and after serials, objects created, request IDs and raw responses
- Every call logged with timestamp, endpoint, method, status and a redacted payload
- Refuse any step whose device or object is not in the plan

Instructor demo.

1. Show the NITRO call sequence for an in-place update against lb-ns-01 in dry-run mode
2. Run it live and show show ssl vserver output before and after
3. Run the Alteon executor with dry_run and show the diff, then apply
4. Run the contract test suite against all three adapters and show the Anaya LB expected failures
5. Open the call log and trace one renewal end to end

Hands-on lab. Implement the NetScaler and Radware executors, prove all three platforms pass the same contract tests, and stub the adapter for other load balancers (Anaya LB).

1. Implement NITRO login, systemfile upload, sslcertkey add or update, binding and save
2. Add a check that refuses writes when the node is not HA primary
3. Implement the Alteon executor against recorded responses with dry-run and diff
4. Write the shared contract test suite
5. Run the suite against F5, NetScaler and Alteon adapters
6. Create the Anaya LB stub and mark its tests expected-fail
7. Run NetScaler live on lb-ns-01 with an approved lab change
8. Review the call log for redaction

Lab success criteria.

- F5, NetScaler and Alteon adapters pass the same contract suite
- lb-ns-01 serves the new serial and the saved config contains the change
- Writes to a secondary node are refused
- Logs contain request IDs and no credentials

Deliverable. Multi-platform executor library with contract tests.

Common pitfalls.

GenNoorTech INNOVATE. BUILD. For more info: https://gennoor.com

- Forgetting nsconfig save, so the change is lost on reboot
- Using add instead of update and leaving bindings on the old pair
- Assuming Alteon changes are live before apply
- Platform-specific fields leaking into the common contract

Knowledge check.

1. Which NITRO call swaps certificate files under existing bindings? Answer: POST /nitro/v1/config/sslcertkey?action=update.
2. Why must NetScaler writes go to the primary? Answer: HA sync propagates from the primary; writes on the secondary are not the source of truth and can be overwritten.
3. What is the purpose of the shared contract tests? Answer: To prove every adapter behaves the same for the pipeline, so new platforms can be added safely.

Slides and assets to prepare.

- NITRO call sequence diagram
- Alteon apply and save flow
- Executor contract specification
- Shared contract test suite

Sources for this module. 2, 3, 4, 15, 16, 17, 29

Day 3

Module 7: Guardrails, approvals and audit for certificate changes

Goal. Learners build a policy module that refuses any install without an approved change, open window, allowed device and healthy HA, and an audit trail that proves it.

Outline topics.

- Policy checks before any write: approved change in ServiceNow, maintenance window open, device on allow list, HA healthy and in sync
- Human approval gate: the approver sees the plan, the diff of bindings and the rollback command; the agent cannot self-approve
- Scope limits: the model proposes, executors refuse anything outside the plan, and credentials never pass through Claude
- Audit log of prompts, tool calls, device responses, approver and timestamps; evidence attached back to the change record

Learning objectives.

- Learners can implement pre-write policy checks against ServiceNow and device state
- Learners can present an approver with plan, binding diff and rollback command
- Learners can ensure the model proposes but cannot approve or execute outside the plan
- Learners can write an audit log and attach evidence to the change

Gennoor Tech TRAIN · INNOVATE · BUILD · For more info: https://gennoor.com

Policy checks before any write

- Change approved: read the change_request and its approval through the Table API [24]
- Window open: compare now with the change's planned start and end
- Device on the allow list for this change; anything else is BLOCKED
- HA healthy and in sync on F5 and NetScaler before writing [13]
- Each check returns PASS or BLOCKED with a reason that is logged

The human approval gate

- Use Ask for Approval on the change record; branch on Approved and treat every other state as no [25]
- The approval view shows the plan, the before-and-after binding diff and the exact rollback command
- The agent's service account has no approval rights, so it cannot approve its own plan
- OWASP recommends human approval for high-impact actions and minimal permissions for agent tools [28]

Scope limits

- Executors check every step against the approved plan hash and refuse anything else
- Hooks block the model from calling executors directly from a chat session [21]
- Credentials are injected into executors at run time and never reach Claude
- Least privilege on devices: executor accounts can manage SSL objects but not users, routing or licensing [28]

Audit trail

- Append-only JSON lines: prompt, tool calls, plan hash, approver, timestamps, device requests and responses
- Attach the evidence bundle to the change record with a work note via PATCH [24]
- Audit entries carry the change number so they can be retrieved per change
- Store the log outside the executor host's write path, for example shipped to a log platform as it is written

Instructor demo.

1. Run a renewal with no change number and show the BLOCKED verdict
2. Run with a change whose window has closed
3. Run with lb-rw-01 not in scope and show the per-device verdict
4. Approve the change in ServiceNow and rerun to PROCEED
5. Open the audit log and the evidence attached to the change

Hands-on lab. Attempt a renewal without an approved change and outside the window, and watch each guardrail block it with a logged reason.

Genoor Tech TRAINING INNOVATE BUILD For more info - https://genoor.com

1. Implement check_change_approved, check_window_open, check_allow_list and check_ha_in_sync
2. Write pytest cases for each check with PASS and BLOCKED fixtures
3. Wire the checks before every executor call
4. Run a renewal with no change number
5. Run with an expired window
6. Run with an unlisted device
7. Approve the change and complete the renewal
8. Export the audit log and attach it to the change

Lab success criteria.

- Each failed condition blocks the run with a specific logged reason
- The approved run proceeds only on in-scope devices
- Audit entries link every call to the change and approver
- No path exists for the agent to approve its own change

Deliverable. Guardrail policy module and audit trail.

Common pitfalls.

- Checking approval once at plan time and not again at execution time
- Time zone mistakes when comparing the window
- Treating Requested or Skipped as approved
- Logs that can be edited after the fact

Knowledge check.

1. Which approval states should allow a write? Answer: Only Approved; all others block.
2. Why re-check HA sync immediately before writing? Answer: State can change between planning and execution; a write to an out-of-sync pair risks serving the old certificate after failover.
3. How does the design stop the agent from approving its own change? Answer: Its service account has no approval rights and approval is read from ServiceNow, not from the agent.

Slides and assets to prepare.

- Guardrail decision table
- Approval screen mock-up showing plan, diff and rollback
- Audit log schema
- Test fixtures for every BLOCKED case

Sources for this module. 13, 21, 24, 25, 28

Gennoor Tech
TRAIN. INNOVATE. BUILD.
For more info: https://gennoor.com

Module 8: Pre- and post-checks, rollback and the GitHub pipeline

Goal. Learners prove every renewal with handshake, chain and serial checks, roll back automatically on failure and ship renewals through pull requests and Actions.

Outline topics.

- Pre-checks: current TLS handshake per virtual server, chain validation, key-certificate match, HA sync status
- Post-checks: new serial served on every virtual server, OCSP and chain valid, no handshake errors; automatic rollback to the previous object on failure
- GitHub pull request per renewal plan with Claude-written summary; GitHub Actions runs pytest, the dry run and the post-check report
- Scheduling renewals ahead of expiry and reporting to the certificate owner

Learning objectives.

- Learners can write pre- and post-checks for handshake, chain, key match, OCSP and serial
- Learners can implement automatic rollback to the previous object
- Learners can run tests, dry run and post-check reports in GitHub Actions
- Learners can schedule renewals ahead of expiry and report to owners

Pre-checks

- Handshake per virtual server with openssl s_client -connect host:443 -servername <fqdn> [20]
- Chain validation with openssl verify -CAfile root.pem -untrusted intermediates.pem leaf.pem [6]
- Key and certificate match by comparing public keys in code before upload
- HA sync status on both pair members

Post-checks and rollback

- New serial served on every bound virtual server and on each HA member after failover test
- OCSP status good with s_client -status where stapling is enabled [20][19]
- Any failure triggers rollback: re-point the profile or binding to the previous date-suffixed object and sync
- Rollback is itself verified with the same post-checks

GitHub pull requests and Actions

- One pull request per renewal plan; Claude writes the summary from plan.json and the evidence
- Actions runs pytest, the dry run and attaches the post-check report as an artifact
- claude -p can generate the summary in CI [8]
- Merging the pull request is not the approval; the ServiceNow approval still gates the install

Scheduling and reporting

Gennoor Tech TRAINING MANUAL for more info - https://gennoor.com

- A schedule workflow runs inventory daily; schedules run in UTC on the default branch and can be delayed at the top of the hour, so pick an off-peak minute [26]
- workflow_dispatch with inputs lets an operator start a renewal run for one change [26]
- Renew well ahead of expiry; with 47-day certificates from 2029 the planning window shrinks to days [18]
- Send the certificate owner the report with serial, expiry, devices and change number

Instructor demo.

1. Run pre-checks on vs_payments_443 and show the report
2. Install a certificate with a broken chain on purpose
3. Show the post-check failing and the executor rolling back
4. Show the rollback verified by the same checks
5. Open a pull request and show Actions running tests, dry run and report

Hands-on lab. Break the chain on purpose, watch the post-check fail and the rollback restore service, then ship a clean renewal through the pull request pipeline.

1. Implement pre-checks and run them on all lab virtual servers
2. Implement post-checks for serial, chain and OCSP
3. Implement rollback to the previous object
4. Install a leaf without its intermediate and watch the post-check fail
5. Confirm rollback restores the old serial
6. Write the Actions workflow with pytest, dry run and report steps
7. Open a pull request for a clean renewal and review the Claude-written summary
8. Run the approved renewal and attach the report to the change

Lab success criteria.

- The broken renewal is rolled back automatically and service is restored
- The clean renewal passes all post-checks on every bound virtual server
- The pull request shows tests, dry run and report
- The owner report is generated

Deliverable. Validation, rollback and CI pipeline.

Common pitfalls.

- Testing without SNI and checking the wrong certificate
- Treating -showcerts output as a verified chain
- Rollback that deletes the new object before re-pointing
- Scheduling at minute zero and seeing delayed runs

Knowledge check.

1. What does -untrusted do in openssl verify? Answer: Supplies intermediates for chain building without treating them as trust anchors.

GenNoorTech TRAIN INNOVATE BUILD For more info - https://gennoor.com

2. What is the rollback target? Answer: The previous date-suffixed certificate object, still present on the device.
3. Does merging the pull request approve the install? Answer: No; the ServiceNow change approval still gates execution.

Slides and assets to prepare.

- Check matrix: pre, post, rollback
- Broken-chain test certificates
- Actions workflow file
- Owner report template

Sources for this module. 6, 8, 18, 19, 20, 26

Module 9: Capstone: end-to-end certificate renewal across the lab estate

Goal. Teams run the full pipeline from inventory to validated renewal on every platform and defend it in a review.

Outline topics.

- Inventory in, approved and validated renewals out on F5, NetScaler, Radware and other load balancers (Anaya LB)
- No install without an approved change and a named approver; every step in the audit log
- Acceptance review, limitations statement and team demo

Learning objectives.

- Learners can run the full pipeline across F5, NetScaler and Alteon
- Learners can show that no install happens without an approved change and a named approver
- Learners can present evidence, limitations and next steps

Capstone brief

- Inventory in, approved and validated renewals out on F5, NetScaler and Alteon, with Anaya LB handled by the stub adapter
- No install without an approved change and a named approver; every step in the audit log
- Teams must show at least one blocked step and one rollback
- Live device steps run only through the GitHub pipeline with an approved lab change

Acceptance review

- Reviewers trace one certificate from inventory row to post-check report
- Reviewers check that every plan step cites evidence
- Teams state limitations, including unverified platforms and what the Anaya LB adapter needs
- Reviewers confirm no key material appears in any transcript, log or pull request

Team demo and handover

- Ten-minute demo: inventory, plan, approval, install, post-check, rollback
- Hand over the operator runbook: how to run, how to stop, how to roll back
- Show the scheduled inventory run and the owner report
- List the next three certificates due and the plan for them

Instructor demo.

1. Show the acceptance checklist and scoring rubric
2. Walk through a sample evidence pack from inventory row to post-check report
3. Show a sample limitations statement
4. Demonstrate how reviewers will trace one certificate through the audit log

Hands-on lab. Capstone build and demo.

1. Run inventory across the estate
2. Generate and review the plan
3. Open the change and get approval
4. Run pre-checks
5. Execute on each platform
6. Run post-checks and handle any rollback
7. Open and merge the pull request with the evidence
8. Present to the group

Lab success criteria.

- All in-scope certificates renewed and verified
- Audit log complete for every step
- At least one guardrail block and one rollback shown
- Limitations statement and operator runbook delivered

Deliverable. End-to-end renewal pipeline and operator runbook.

Common pitfalls.

- Spending the time on code polish instead of evidence
- Skipping the blocked and rollback cases
- Not stating limitations

Knowledge check.

1. What must every renewal in the capstone have? Answer: An approved change, a named approver, pre- and post-check evidence and audit entries.
2. What is the Anaya LB position at the end of the course? Answer: An adapter stub that passes the contract shape but needs the client's API details before it can write.
3. What proves the pipeline cannot install without approval? Answer: A demonstrated BLOCKED run with a logged reason, and executors that read approval state from ServiceNow at run time.

Gennoor Tech AI · INNOVATE · BUILD
For more info - https://gennoor.com

Slides and assets to prepare.

- Capstone acceptance checklist
- Scoring rubric
- Operator runbook template

Sources for this module. 18, 24, 25

6. Capstone plan

A repeatable pipeline that discovers expiring certificates across F5, NetScaler, Radware and other load balancers (Anaya LB), has Claude draft a cited renewal plan and ServiceNow change, installs through deterministic tested executors only after human approval, validates every binding, rolls back on failure and leaves a complete audit trail.

Teams build and demonstrate a repeatable pipeline that discovers expiring certificates across F5, NetScaler, Radware and the Anaya LB adapter, has Claude draft a cited renewal plan and ServiceNow change, installs through tested executors only after human approval, validates every binding, rolls back on failure and leaves a full audit trail.

Milestones

1. Inventory and expiry report across the estate
2. Validated plan with citations and at least one blocked step
3. Approved change in ServiceNow
4. Executed renewals with pre- and post-check evidence
5. One deliberate failure rolled back
6. Pull request with evidence and owner report

Evaluation notes

- Score evidence and safety above code style
- Every install must trace to an approved change and named approver
- Check that no key material appears in transcripts or logs
- Limitations statement must cover Anaya LB and any platform run only against recorded responses

Assessment weights

Gennoor Tech
For more info - https://gennoor.com

Assessment area	Weight	Evidence
Working functionality	25%	Demonstrable end-to-end workflow in a repeatable lab
Security and safety controls	20%	Threat model, authorization boundaries, approval gates, safe failure
Testing and evaluation	20%	Automated tests, golden dataset, measured quality and known limitations
Engineering quality	15%	Clear interfaces, error handling, reproducible setup, maintainable code
Evidence and explainability	10%	Traceable findings, sources, assumptions, logs
Documentation and presentation	10%	Architecture diagram, README, runbook, demo, lessons learned

7. Promotional video: research behind the script

Opening scenario. The midnight-expiry hook is a teaching scenario, not a reported incident. It is realistic because NetScaler's own expiry monitoring only alerts and does not renew [2], bindings are per virtual server [3], and certificate lifetimes are shrinking toward 47 days, which multiplies renewal events [18].

GenMoorTech TRAIN · INNOVATE · BUILD
For more info - https://genmoor.com

Claim or on-screen text	Basis
4 load balancer platforms: F5, NetScaler, Radware and other load balancers (Anaya LB)	F5, NetScaler and Radware are covered from vendor sources [1][2][29]. Anaya LB is the client's named platform; we found no public documentation, so it is covered by an adapter stub only.
Multi-platform inventory through iControl, NITRO and Alteon	F5 ssl-cert and client-ssl endpoints [1][10], NITRO sslcertkey and binding resources [2][15], Alteon through the Radware SDK over REST [29].
Terminal: lb-f5-01 41 certs, 3 expiring; pay.example.corp expires 2026-11-19; owner UNKNOWN blocks the plan	Illustrative lab output with fictional hosts and counts. The blocking behavior is a course design rule, not a vendor feature.
Planning agent: structured JSON, cited rows	Structured outputs constrain responses to a JSON schema [23]; citation of inventory rows is enforced by the course validator.
ServiceNow change via Table API and Flow Designer; CHG0041218 created, state assess	Change records are created with POST /api/now/table/change_request [24] and routed through Ask for Approval [25]. The change number and state label are illustrative.
CSR generated by executor, key never shown to the model	Course design using standard CSR generation in code [27]; the CA only receives the CSR [27][5].
F5 through iControl REST and tmsh; NetScaler with primary-only writes and saved config; Radware with apply and save as explicit steps	F5 file upload and objects [12][1][11]; NITRO save [17]; Alteon apply step [4]. Primary-only writes and explicit save on Alteon are course design rules.
Guardrails: approved change, open window, device on allow list, HA in sync; verdict PROCEED or BLOCKED	Course policy module using ServiceNow data [24][25] and F5 sync state [13]; hostnames and approver are fictional.
Post-check: new serial served 2/2 nodes, OCSP good	Serials and stapled OCSP status can be read with openssl s_client -showcerts and -status [20]; good is a defined OCSP status [19].
Rollback to previous object, tested; PR #184 merged, pytest 96 passed, audit entries 23	Rollback to a date-suffixed previous object is the course design; PR number and counts are illustrative.
9 build steps; 3 days; 0 installs without approval	Matches the nine modules over three days in the outline; zero unapproved installs is the guardrail design, demonstrated in labs.
Never get paged for an expired certificate again	Marketing call to action, not a factual claim; the pipeline reduces but cannot remove the risk, for example for certificates outside inventory.

Gemnoor Tech
For more info - https://gemnoor.com

References

[1] APIRef_tm_sys_file_ssl-cert. F5 (clouddocs.f5.com). https://clouddocs.f5.com/api/icontrol-rest/APIRef_tm_sys_file_ssl-cert.html
Supports: Inventory; F5 executor. The /mgmt/tm/sys/file/ssl-cert collection supports GET; individual resources support GET, POST, PUT, PATCH and DELETE, are keyed by name, partition and subPath, take a sourcePath (the URI the file is copied from) and expose read-only X.509 details including expirationDate, expirationString, subject, issuer, subjectAlternativeName, serialNumber and fingerprint.

[2] sslcertkey | configuration (NITRO API). NetScaler (developer-docs.netscaler.com). https://developer-docs.netscaler.com/en-us/adc-nitro-api/current-release/configuration/ssl/sslcertkey.html
Supports: Inventory; NetScaler executor. Add is POST /nitro/v1/config/sslcertkey with mandatory certkey and cert; update is PUT with certkey plus expirymonitor and notificationperiod (10 to 100 days); the change operation is POST /nitro/v1/config/sslcertkey?action=update and accepts nodomaincheck; daystoexpiration is a read-only property; linkcertkeyname links a CA cert-key pair. Sessions start at /nitro/v1/config/login and use Cookie: NITRO_AUTH_TOKEN, or per-request X-NITRO-USER and X-NITRO-PASS headers.

[3] Create a certificate signing request and use SSL certificates on a NetScaler appliance. NetScaler (docs.netscaler.com). https://docs.netscaler.com/en-us/citrix-adc/current-release/ssl/how-to-articles/create-and-use-ssl-certificates-on-a-citrix-adc-ap.html
Supports: Lifecycle; NetScaler executor. Certificates and keys live in /nsconfig/ssl by default; add ssl certKey <name> -cert <file> -key <file> [-expiryMonitor ENABLED -notificationPeriod <days>] creates the pair; intermediate certificates must be linked before binding; every SSL virtual server needs its own valid binding, made with bind ssl vserver <vs> -certkeyName <name> and checked with show ssl vserver <vs>.

[4] SSL Certificate Installation: Radware Alteon. DigiCert Knowledge Base. https://knowledge.digicert.com/tutorials/ssl-certificate-installation-radware-alteon
Supports: Lifecycle; Radware executor. On Alteon the installation flow is to build one PEM file with the server certificate followed by the intermediate chain, enter cfg then cert on the CLI, paste the PEM, end the input with three periods and then run apply; the page does not describe a separate save step (no public Radware page with the same detail could be fetched).

[5] RFC 8555: Automatic Certificate Management Environment (ACME). IETF (rfc-editor.org). https://www.rfc-editor.org/rfc/rfc8555
Supports: Renewal planning; lab CA. After registering an account an ACME client submits an order, proves control of each identifier through a challenge (for example http-01 or dns-01), finalizes the order by submitting a PKCS#10 CSR and then downloads the issued certificate.

[6] openssl-verify. OpenSSL Project. https://docs.openssl.org/master/man1/openssl-verify/
Supports: Pre- and post-checks. openssl verify checks certificate chains; -CAfile supplies trusted anchors, while -untrusted (repeatable) supplies intermediate certificates used only for chain building.

[7] What is the Model Context Protocol (MCP)?. Model Context Protocol. https://modelcontextprotocol.io/docs/getting-started/intro
Supports: Workbench setup. MCP is an open standard, described as a USB-C port for AI applications, that gives AI applications a standard way to connect to external data sources, tools and workflows.

[8] Claude Code overview. Anthropic. https://code.claude.com/docs/en/overview
Supports: Workbench setup; GitHub pipeline. Claude Code is an agentic coding tool for the terminal, IDE, desktop and browser; on macOS, Linux and WSL it installs with curl -fsSL https://claude.ai/install.sh | bash, and it can run non-interactively with claude -p for piping and CI use.

[9] Agent SDK overview. Anthropic. https://code.claude.com/docs/en/agent-sdk/overview
Supports: Workbench setup; planning agent. The Agent SDK is a Python or TypeScript library that runs the Claude Code engine with its built-in tools, permissions, sessions and hooks; products built on it must use API-key authentication rather than claude.ai logins unless previously approved.

[10] APIRef_tm_ltm_profile_client-ssl. F5 (clouddocs.f5.com). https://clouddocs.f5.com/api/icontrol-rest/APIRef_tm_ltm_profile_client-ssl.html
Supports: Inventory; F5 executor. Client SSL profiles live at /mgmt/tm/ltm/profile/client-ssl (resource methods GET, PUT, PATCH, DELETE, POST; key name, partition, subPath); the cert and chain properties are DEPRECATED in favor of certKeyChain, and sniDefault marks the profile used when the client sends no known server name.

[11] ltm profile client-ssl (tmsh reference). F5 (clouddocs.f5.com). https://clouddocs.f5.com/cli/tmsh-reference/latest/modules/ltm/ltm_profile_client-ssl.html
Supports: F5 executor; verification. In tmsh, cert-key-chain is changed with add, delete, modify or replace-all-with, each entry naming cert, chain, key, passphrase and usage; a client-ssl profile requires at least one cert/key pair to work.

[12] Demystifying iControl REST Part 5: Transferring Files. F5 DevCentral. https://community.f5.com/kb/technicalarticles/demystifying-icontrol-rest-part-5-transferring-files/286689
Supports: F5 executor. POST /mgmt/shared/file-transfer/uploads/ writes files to /var/config/rest/downloads/; transfers are chunked with a Content-Range: start-end/filesize header, and any file larger than 1M must be chunked because the worker enforces that limit.

[13] cm config-sync (tmsh reference). F5 (clouddocs.f5.com). https://clouddocs.f5.com/cli/tmsh-reference/latest/modules/cm/cm_config-sync.html
Supports: F5 executor; guardrails. run cm config-sync starts a configuration synchronization job and must be given one of to-group, from-group or recover-sync to set the direction (force-full-load-push is also available).

[14] APIRef_tm_sys_file_ssl-key. F5 (clouddocs.f5.com). https://clouddocs.f5.com/api/icontrol-rest/APIRef_tm_sys_file_ssl-key.html
Supports: F5 executor. Private keys are managed at /mgmt/tm/sys/file/ssl-key; resources support GET, PUT, PATCH, DELETE and POST and are keyed by name, partition and subPath.

[15] sslvserver_sslcertkey_binding (NITRO API). NetScaler (developer-docs.netscaler.com). https://developer-docs.netscaler.com/en-us/adc-nitro-api/current-release/configuration/ssl/sslvserver_sslcertkey_binding.html
Supports: Inventory; NetScaler executor. Bindings are added with PUT /nitro/v1/config/sslvserver_sslcertkey_binding (vservername mandatory, plus certkeyname, ca, snicert and ocspcheck) and read with GET /nitro/v1/config/sslvserver_sslcertkey_binding/<vservername>.

[16] systemfile (NITRO API). NetScaler (developer-docs.netscaler.com). https://developer-docs.netscaler.com/en-us/adc-nitro-api/current-release/configuration/system/systemfile.html
Supports: NetScaler executor. POST /nitro/v1/config/systemfile uploads a file with filename (no path), filecontent in Base64 and filelocation; fileencoding defaults to BASE64.

[17] nsconfig (NITRO API). NetScaler (developer-docs.netscaler.com). https://developer-docs.netscaler.com/en-us/adc-nitro-api/current-release/configuration/ns/nsconfig.html
Supports: NetScaler executor. The running configuration is saved with POST /nitro/v1/config/nsconfig?action=save.

[18] Baseline Requirements for TLS Server Certificates. CA/Browser Forum. https://cabforum.org/working-groups/server/baseline-requirements/requirements/
Supports: Why this course; lifecycle; scheduling. Maximum validity for publicly trusted TLS subscriber certificates is 398 days before 2026-03-15, 200 days from 2026-03-15, 100 days from 2027-03-15 and 47 days from 2029-03-15.

[19] RFC 6960: X.509 Internet PKI Online Certificate Status Protocol (OCSP). IETF (rfc-editor.org). https://www.rfc-editor.org/rfc/rfc6960
Supports: Pre- and post-checks. OCSP responses report a certificate status of good, revoked or unknown, with thisUpdate and nextUpdate times; good means no certificate with that serial number in its validity interval is known to be revoked.

[20] openssl-s_client. OpenSSL Project. https://docs.openssl.org/master/man1/openssl-s_client/
Supports: Pre- and post-checks. openssl s_client -connect host:port opens a TLS connection; -servername sets

SNI, -showcerts prints the certificates the server actually sent (not a verified chain) and -status requests an OCSP stapled response.

[21] Hooks reference. Anthropic. https://code.claude.com/docs/en/hooks
Supports: Workbench setup; guardrails. PreToolUse hooks run after Claude creates tool parameters and before the call is processed, match built-in and MCP tool names, and can block the call (exit code 2) or return permissionDecision deny with a reason.

[22] Give Claude custom tools (Agent SDK). Anthropic. https://code.claude.com/docs/en/agent-sdk/custom-tools
Supports: Workbench setup. Python tools are defined with @tool, wrapped with create_sdk_mcp_server and passed in mcp_servers; their full names are mcp__{server_name}__{tool_name} and are pre-approved through allowed_tools; tools can set readOnlyHint, return structuredContent and return isError: true on failure.

[23] Structured outputs. Anthropic. https://platform.claude.com/docs/en/build-with-claude/structured-outputs
Supports: Planning agent. JSON outputs send a JSON schema in output_config.format (type json_schema) so the response matches the schema, and strict tool use (strict: true) guarantees schema validation of tool names and inputs.

[24] Table API. ServiceNow. https://www.servicenow.com/docs/r/api-reference/rest-apis/c_TableAPI.html
Supports: Planning agent; guardrails. The Table API provides GET, POST, PUT, PATCH and DELETE on /api/now/table/{tableName} (and /{sys_id}); POST inserts one record and returns 201; GET filters with sysparm_query encoded queries and trims fields with sysparm_fields.

[25] Ask for Approval action. ServiceNow. https://www.servicenow.com/docs/r/build-workflows/workflow-studio/ask-approval-flow-designer.html
Supports: Planning agent; guardrails. Ask for Approval takes a record, table, approval and rejection rules and an optional due date, and outputs an approval state (Requested, Approved, Rejected, Cancelled, No Longer Required, Skipped) that later flow steps can branch on.

[26] Events that trigger workflows. GitHub Docs. https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows
Supports: GitHub pipeline. schedule runs on the latest commit of the default branch in UTC (optional IANA timezone), no more often than every 5 minutes, and may be delayed or dropped at high load such as the top of the hour; workflow_dispatch accepts typed inputs and needs the workflow file on the default branch.

[27] X.509 Tutorial. pyca/cryptography. https://cryptography.io/en/latest/x509/tutorial/
Supports: Planning agent; executors. x509.CertificateSigningRequestBuilder builds and signs a CSR in Python; the usual flow is to generate a key pair, sign a CSR with it and give the CA the CSR but never the private key.

[28] LLM06:2025 Excessive Agency. OWASP Gen AI Security Project. https://genai.owasp.org/llmrisk/llm062025-excessive-agency/
Supports: Guardrails. Mitigations include minimizing extension functionality and permissions and requiring human-in-the-loop approval of high-impact actions, implemented in a downstream system or the extension itself.

[29] alteon-sdk (Python package metadata). PyPI (package copyright Radware Ltd). https://pypi.org/pypi/alteon-sdk/json
Supports: Inventory; Radware executor. alteon-sdk wraps the Alteon REST backend with 46 configurators (including SSLKeyConfigurator), management functions such as config commit, and deploy options dry_run, write_on_change and get_diff; minimum supported Alteon versions are 31.0.10.0 and 32.2.2.0.

[30] How Claude remembers your project (CLAUDE.md). Anthropic. https://code.claude.com/docs/en/memory
Supports: Workbench setup. A project CLAUDE.md lives at ./CLAUDE.md or ./.claude/CLAUDE.md, is shared through source control and should hold build and test commands, coding standards, naming conventions and common workflows.

[31] ACME Basics (step-ca). Smallstep. https://smallstep.com/docs/step-ca/acme-basics/
Supports: Lab environment; planning agent. step-ca enables ACME with step ca provisioner add acme --type ACME and supports http-01, dns-01 and tls-alpn-01 challenges; after challenges succeed the client submits a CSR and the CA issues the certificate.

[32] NetScaler licensing (VPX Express). NetScaler (docs.netscaler.com). https://docs.netscaler.com/en-us/citrix-adc/current-release/licensing
Supports: Lab environment. VPX Express needs no license file and offers 20 Mbps bandwidth, a maximum of 250 SSL sessions and 20 Mbps SSL throughput; from release 14.1 build 29.65 it includes all Premium-license features.

[33] Understanding PDIs. ServiceNow.
https://www.servicenow.com/docs/r/SmYvGPN0OygSQs53fsexIQ/yG_RdLKb649OcfnDiGEkwQ
Supports: Lab environment. Personal Developer Instances are free for Developer Program members; an instance unused for ten days is reclaimed and reset, and idle instances hibernate until the owner signs in to the Developer Site.

For more info - https://gennoor.com

Gennoor Tech
TRAIN · INNOVATE · BUILD
For more info - https://gennoor.com